CIT300 Data structures and Algorithms
Graded Practical Assignment 2 (Week 12)
Data Structure and Graph Performance Analyzer
Scope: Weeks 1-11
Programming Language: Java
Application Type: Console-Based System
Submission: GitHub Repository + One Merged Demonstration Video
Contribution: 10% of the final grade

1. Assignment Objective

The objective of this assignment is to develop a **Java-based console application** that demonstrates the practical application of data structures, algorithms, searching, graph concepts, and algorithmic complexity.

The system must allow users to work with different data structures and perform relevant operations while demonstrating how different algorithms and data structures can be used to solve computational problems.

Students are expected to demonstrate:

- Correct implementation of data structures
- Appropriate use of algorithms
- Searching and traversal
- Graph representation and operations
- Performance/operation comparison
- Input validation
- Object-oriented programming principles
- GitHub-based collaboration
- Individual understanding and contribution

2. System Requirements

Each group must develop a single integrated console-based application.

The application must include **ALL** of the following components:

A. Array

Implement appropriate operations such as:

- Insert
- Delete
- Search
- Display

B. Stack

Implement:

- Push
- Pop
- Peek
- Display

The system should handle conditions such as attempting to pop from an empty stack.

C. Queue

Implement:

- Enqueue
- Dequeue
- Peek/Front
- Display

The system should handle conditions such as attempting to dequeue from an empty queue.

D. Linked List

Implement appropriate operations such as:

- Insert
- Delete
- Search
- Display

E. Searching

Implement appropriate searching functionality.
For example:

- Linear Search
- Binary Search

Where appropriate, the program should allow the user to observe the difference between the searching approaches.

F. Graph Component

The graph component is **mandatory**.

Each group must implement a graph using a suitable representation and provide appropriate graph functionality.

The graph component should include suitable operations such as:

- Add vertex
- Add edge
- Display graph
- Search/traverse the graph
- BFS traversal
- DFS traversal

The implementation should demonstrate the students' understanding of graph data structures and traversal algorithms.

The graph component must not be omitted, even if the group has fewer members.

3. Example Main Menu

```
==============================================
      DATA STRUCTURE & GRAPH ANALYZER
==============================================
1. Array Operations
2. Stack Operations
3. Queue Operations
4. Linked List Operations
5. Searching Operations
6. Graph Operations
7. Performance Comparison
8. Display All Results
9. Exit
Enter your choice:
```

Each component should have an appropriate submenu.

For example:

```
---------- GRAPH OPERATIONS ----------

1. Add Vertex
2. Add Edge
3. Display Graph
4. BFS Traversal
5. DFS Traversal
6. Return to Main Menu
Enter your choice:
```

The menu structure may be modified by the group provided that all required functionality is implemented.

4. Performance / Complexity Demonstration

The system should demonstrate the performance or operation requirements of appropriate algorithms and data structures.

Where applicable, students should record:

- Number of operations/steps
- Execution time
- Search results

- Traversal results
- Other meaningful performance information

The program may display results in a format such as:

```
==============================================
        PERFORMANCE COMPARISON
==============================================

Operation         Algorithm       Steps
----------------------------------------------

Search            Linear Search   XXXXX
Search            Binary Search   XXXXX
Graph Traversal   BFS             XXXXX
Graph Traversal   DFS             XXXXX
==============================================
```

Students must be able to explain **why** the results may differ and relate their explanation to algorithmic complexity.

5. Important Group Requirement

Every group must complete the **entire project**.

If your group has fewer members, combine the suggested responsibilities among the available members and still implement **all specified components, including the graph component**.

No component may be omitted because the group has fewer members.

Group-member details must be recorded correctly.

The **project folder and README** must include:

- Full name of every member
- Student ID of every member
- Assigned responsibility
- Individual contribution

Missing, incomplete, or incorrect information may result in marks being deducted.

6. Suggested Team Responsibilities

The following are suggested responsibilities only.

Member	**Suggested Responsibility**
Member 1	Array and Searching implementation
Member 2	Stack and Queue implementation
Member 3	Linked List implementation

Member 4	Graph implementation and traversal
Member 5 / Integration	Performance comparison, main menu, integration and testing

These are suggested roles only.

If a group contains fewer members, the responsibilities must be combined.

For example, a four-member group may combine the integration responsibility with another member's assigned component.

Important

Regardless of the number of members:

Every required component must be implemented.

This includes:

- Array
- Stack
- Queue
- Linked List
- Searching
- Graph
- Performance/complexity demonstration
- Main system integration

7. Individual Contribution

Each member must make a genuine contribution to the project.

The README must clearly identify:

Student Name:
Student ID:
Assigned Responsibility:
Individual Contribution:
Example:
Student Name: ABC Perera
Student ID: IT12345678
Assigned Responsibility: Graph Component

Individual Contribution:

- Implemented Graph class
- Implemented vertex and edge operations
- Implemented BFS traversal
- Implemented DFS traversal
- Tested graph functionality

- Integrated graph module with the main application

All members must understand their own contribution and be able to explain it during the demonstration.

8. Implementation Guidelines

- Develop the application using **Java** as a console-based system.
- Implement the required data structures using appropriate classes and methods.
- Do not omit any required component or the graph functionality.
- Use meaningful variable, method, and class names.
- Write readable and well-structured code.
- Include suitable comments where necessary.
- Validate user inputs.
- Handle invalid operations appropriately.
- Handle empty data structures appropriately.
- Include a README with complete project information.
- Include complete and correct group-member details.
- Clearly document individual responsibilities and contributions.
- Each member must be able to explain and demonstrate their own contribution.
- Use GitHub collaboration features such as branches, commits, and pull requests where applicable.

9. GitHub Collaboration Requirement

The complete project must be maintained in a GitHub repository.

The repository should provide evidence of genuine collaboration.

Where applicable, students should use:

- Individual branches
- Meaningful commits
- Pull requests
- Merging
- Issue/task tracking where appropriate

The lecturer may inspect the GitHub repository to verify individual contributions.

Students should not simply upload the final project with no evidence of individual development.

10. Submission and Google Drive Access Requirement

If the project files are too large to upload directly to the designated LMS submission link, upload the **complete project** to a Google Drive folder.

Then:

1. Copy the Google Drive folder link.
2. Create a Notepad (.txt) file.

3. Paste the Google Drive folder link into the TXT file.
4. Upload the TXT file through the designated LMS submission link.

Google Drive Permissions

Before submitting, you **MUST give Editor access** to both of the following email addresses:

- **asanka.r@sltc.ac.lk**
- **kaushika.w@sltc.ac.lk**

Check the Google Drive sharing settings before clicking **Submit** in the LMS.
Confirm that **both email addresses have Editor access**.
Students are responsible for verifying the permissions before submission.

Important

Failure to provide the required Google Drive Editor permissions before submission may result in **0 marks**.

11. Late Submission Policy

Late emails will **not** be accepted.

Even if you send an email after the deadline containing:

- Required project files
- Google Drive links
- Google Drive permissions
- Access permissions
- Other submission information

such submissions will **not** be considered or accepted.

Only submissions successfully submitted through the **designated LMS submission link before the deadline** will be considered for marking.

An email sent after the deadline does **not** make a late submission valid.

12. Demonstration Video

Each group must submit **ONE merged demonstration video**.

Maximum Duration

Less than 15 minutes

The video should demonstrate the completed integrated system.

Face Visibility Requirement

ALL MEMBERS MUST ENSURE THAT THEIR FACES ARE CLEARLY VISIBLE THROUGHOUT THEIR RESPECTIVE PRESENTATION.

Each member must appear on camera while presenting their contribution.

A recording that shows only:

- The computer screen
- The source code
- The program output
- Voice without the student's face

will **not be sufficient** as evidence of that student's individual demonstration.

13. Required Video Content

The merged video should contain a section for each group member.

Each member should:

1. Introduce themselves

Briefly state:

- Name
- Student ID
- Assigned responsibility

2. Explain their contribution

Briefly explain what they implemented.

3. Show their code

Show the relevant classes, methods, or sections of the implementation.

4. Demonstrate the functionality

Run the program and demonstrate their implemented functionality.

5. Explain the concept

Explain the relevant:

- Data structure
- Algorithm
- Operations
- Complexity/performance

6. Demonstrate integration

Where appropriate, show how their component works as part of the complete system.

14. Example Individual Video Section

Each student's section should approximately follow:

Student's Face Visible
↓
Introduction
↓
Assigned Responsibility
↓
Relevant Code
↓
Program Demonstration
↓
Explanation of Algorithm/Data Structure
↓
Complexity / Performance Explanation
↓
Integration with Complete System
The sections can then be merged into **one final video of less than 15 minutes**.

15. Deliverables

Each group must submit:

1. GitHub Repository

The repository must contain the complete Java project.

2. README

The README must include:

- Project title
- Project description
- Team member names
- Student IDs
- Assigned responsibilities
- Individual contributions
- Technologies used
- Main system features
- Instructions for running the program

3. One Merged Demonstration Video

- Less than 15 minutes
- Complete system demonstrated
- Contribution of every member represented
- **Faces of all members clearly visible throughout their respective presentation**

4. Collaboration Evidence

The GitHub repository should contain evidence such as:

- Commits
- Branches
- Pull requests

where applicable.

5. Google Drive TXT File - If Required

If the project is too large for direct LMS upload:

- Upload the complete project to Google Drive.
- Copy the folder link into a .txt file.
- Upload the TXT file to the designated LMS submission link.
- Give Editor access to both required email addresses.

16. Final Submission Checklist

Before clicking **Submit** in the LMS, make sure that all of the following have been completed.

☐ Complete Java project implemented.
☐ Array component completed.
☐ Stack component completed.
☐ Queue component completed.
☐ Linked List component completed.
☐ Searching functionality completed.
☐ **Graph component completed.**
☐ BFS/DFS or required graph traversal functionality completed.
☐ Performance/complexity demonstration completed.
☐ Main console application integrated.
☐ All group members' names are correct.
☐ All student IDs are correct.
☐ Responsibilities are documented.
☐ Individual contributions are documented.
☐ GitHub repository is complete.
☐ README is complete and accurate.
☐ GitHub commits/branches/pull requests provide collaboration evidence where applicable.
☐ One merged demonstration video is prepared.
☐ Video is **less than 15 minutes**.
☐ Every member's contribution is represented in the video.
☐ **Every member's face is clearly visible throughout their respective presentation.**
☐ If using Google Drive, the complete project has been uploaded.

☐ Google Drive folder link has been copied correctly into the Notepad (.txt) file.
☐ **asanka.r@sltc.ac.lk** has Editor access.
☐ **kaushika.w@sltc.ac.lk** has Editor access.
☐ Google Drive permissions have been checked.
☐ LMS submission link has been used.
☐ Submission has been successfully completed before the deadline.

17. Deadline

- **Submit through the designated LMS submission link on or before 29th September.**
- **Important:** An email sent after the deadline does not make a late submission valid, even when files, links, permissions, or access are provided by email.
- Only submissions successfully submitted through the designated LMS submission link before the deadline will be considered for marking.

Important Reminder

- **All groups must implement the complete project.**
- Having fewer group members does **not** allow any component to be removed.
- **Mandatory components:**
- **Array + Stack + Queue + Linked List + Searching + Graph + Performance/Complexity + Integrated Console Application**
- Every member must contribute, demonstrate, and be able to explain their own work.
- **The face of every member must be clearly visible throughout their respective video presentation.**